

WYCKOFFTRADERPRO

Cloud Engineering & Trading Automation Portfolio

Personal project case study

Serverless processing • Firestore • Automation • Monitoring • Troubleshooting

Prepared by: WaNZuL-Pro

GitHub: <https://github.com/WaNZuL-Pro/WyckoffTraderPro-Portfolio>

LinkedIn: [Optional – add URL later]

1. Project Overview

WyckoffTraderPro is a personal cloud-based trading-engineering project designed to collect market data, detect market structure, generate trading signals and support paper-trading analysis. The project combines cloud operations, JavaScript/Node.js development, Firestore data handling, scheduled automation, monitoring and systematic troubleshooting.

Portfolio scope: This document presents engineering work and learning outcomes. Trading objectives, strategy results and backtesting figures are analysis context only and are not guarantees of future performance.

2. My Role & Responsibilities

As the project owner and developer, WaNZuL-Pro is responsible for designing, developing, operating and improving the overall workflow.

- Designed and developed the cloud-based trading-engineering workflow.
- Developed and maintained JavaScript/Node.js functions and supporting scripts.
- Integrated market-data, paper-trading and Telegram APIs.
- Designed Firestore data flow for candles, signals, trades and system state.
- Investigated runtime errors, scheduler issues, duplicate signals and order synchronisation problems.
- Applied cost-conscious data retrieval, caching, batching, retry and cleanup approaches.
- Performed backtesting, shadow analysis and live paper-trading observation.
- Documented architecture, operational issues, lessons learned and improvement plans.

3. Skills Demonstrated

Skill area	Demonstrated capability
Cloud & infrastructure	Google Cloud Functions, Cloud Run runtime, Firebase ecosystem, Cloud Scheduler and Cloud Logging.
Development	JavaScript / Node.js, Python for selected analysis and data-processing tasks, REST API integration and async processing.
Data & database	Cloud Firestore, candle storage, signal/trade lifecycle, caching and targeted queries.
Reliability & operations	Runtime troubleshooting, log analysis, retries, backoff, rate-limit handling and synchronisation.
Automation	Scheduled workflows, market-window controls, polling and operational notifications.
Trading-system engineering	Market-data processing, setup evaluation, signal generation, paper trading, backtesting and shadow analysis.
Version control	Git, GitHub, private source repository and public documentation repository.

4. High-Level Architecture

Market Data API → Scheduled Cloud Function → Cloud Firestore → Recent-Candle Cache / Structure Engine → Decision Engine → Signals → Paper Execution → Order Synchronisation → Reports / Telegram

Layer	Responsibility
Market data	Incremental candle retrieval, historical backfill and rate-limit handling.

Cloud execution	Scheduled serverless functions and managed runtime execution.
Data layer	Firestore collections for candles, signals, trades, state and operational records.
Processing	Recent-candle cache, structure detection, setup evaluation and decision logic.
Execution workflow	Paper-trading actions, order-status polling and trade lifecycle updates.
Operations	Cloud Logging, Telegram notifications, cleanup and cost monitoring.

5. System Design Details

5.1 Multi-Timeframe Processing

The project uses multiple timeframes to separate market context from signal-level analysis. The 1H timeframe is used for signal and setup evaluation, while the 4H timeframe provides broader structure and trend context where applicable.

5.2 Recent-Candle Cache

A recent-candle cache is used to retain sufficient candle context for structure and setup processing. This reduces the need to repeatedly retrieve large historical datasets during scheduled execution.

5.3 Scheduled Workflow

The scheduled workflow is designed to check the market window, retrieve the latest candles, update recent data, run structure and decision processing, and produce signal or operational outputs.

5.4 Firestore Data Model

Collection / data area	Purpose
bars	Market candle data.
bars_tail_cache	Recent candle context for processing.
live_engine_signals	Signal and decision output records.
papertrade_signals	Signals routed to the paper-trading workflow.
papertrade_trades	Trade lifecycle and execution records.
system_state	System state and operational control information.

6. Engineering Challenges & Troubleshooting

Case Study 1 — Runtime TypeError

Issue: A runtime error was encountered around `formatEngineTrade` being unavailable as a function.

Investigation: Reviewed runtime logs, function exports/imports, module structure and the failing execution path.

Engineering lesson: Serverless functions require clear module boundaries, correct exports and runtime validation.

Case Study 2 — Missed Signals / Duplicate Signals

Issue: Signals were not always processed as expected, including duplicate-signal conditions.

Investigation: Compared live signal records with paper-trading signal records, reviewed signal identifiers and traced the signal lifecycle.

Engineering lesson: Event-driven workflows benefit from deterministic identifiers, idempotency and clear state transitions.

Case Study 3 — Telegram Notification Failure

Issue: Telegram notifications encountered HTTP errors and message-length limitations.

Investigation: Reviewed payload size, active-trade detail output and notification formatting.

Engineering lesson: External API integrations require payload validation, size control and controlled error handling.

Case Study 4 — Scheduler / Timeout Problems

Issue: Scheduled jobs and order-status synchronisation encountered timeout or network-related failures.

Investigation: Reviewed execution logs, polling behaviour, retry handling and safe re-execution considerations.

Engineering lesson: Cloud jobs should use bounded retries, timeout awareness and idempotent processing.

7. Cost-Optimisation Practices

- Incremental data retrieval instead of repeated full historical downloads.
- Historical backfill in controlled batches.
- Batch processing and batched Firestore writes.
- Rate limiting and HTTP 429 handling.
- Bounded retry and exponential backoff for transient failures.
- Recent-candle tail caching.
- Deterministic candle identifiers.
- Targeted queries for active and due trades.
- Polling controls and terminal-state filtering.
- Retention and housekeeping to control data growth.
- Budget alerts and service-level cost review.

8. Monitoring & Operational Visibility

- Cloud runtime logs for scheduled function execution.
- Error investigation through Cloud Logging.
- Signal and trade status records in Firestore.
- Telegram notifications for operational and trading events.
- Diagnostic statuses for rejected, duplicate or incomplete processing.
- Review of scheduler, polling and synchronisation behaviour.

9. Portfolio Evidence — Screenshots to Add

Figure 1 — Cloud deployment

The screenshot shows the Firebase Functions dashboard. On the left is a sidebar with navigation links: Search for products, Project Overview, Settings, Project shortcuts (Functions, Firestore, Storage), Product categories (Databases & Storage, Security, AI services, Hosting & Serverless, DevOps & Engagement, Analytics), and Blaze (Pay as you go, Modify). The main area is titled 'Functions' and has tabs for 'Dashboard' and 'Usage'. A notification banner at the top says 'Set up spending caps to pause service usage when costs reach your designated threshold.' Below this is a table of functions:

Function	Trigger	Version	Requests (24 hrs)	Min / Max Instances	Timeout
engineSignalTelegram us-central1	document.created	v2	37	0 / 20	1m
papertradeOrderStatusSync us-central1	every 5 minutes	v2	285	0 / 20	5m
adminMockActiveTrade us-central1	HTTP Request https://adminmockactivetrade-prjwoc3oa-uc.a.run.app	v2	0	0 / 20	1m
syncBarsOnce us-central1	HTTP Request https://syncbarsonce-prjwoc3oa-uc.a.run.app	v2	0	0 / 20	1m
adminResetSyncState us-central1	HTTP Request https://adminresetstate-prjwoc3oa-uc.a.run.app	v2	0	0 / 20	1m
adminSyncRecent us-central1	HTTP Request https://adminsyncrecent-prjwoc3oa-uc.a.run.app	v2	0	0 / 20	1m
adminSetTelegramBotCommands us-central1	HTTP Request https://adminsettelegrambotcommands-prjwoc3oa-uc.a.run.app	v2	0	0 / 20	1m
papertradeSignalTelegram us-central1	document.created	v2	1	0 / 20	1m
adminBackfillHistory us-central1	HTTP Request https://adminbackfillhistory-prjwoc3oa-uc.a.run.app	v2	0	0 / 20	1m
adminTelegramTestFullCycle us-central1	HTTP Request https://admintelegramtestfullcycle-prjwoc3oa-uc.a.run.app	v2	0	0 / 20	1m
adminTestPapertrade us-central1	HTTP Request https://admintestpapertrade-prjwoc3oa-uc.a.run.app	v2	0	0 / 20	1m

Figure 2 — Runtime logs

The screenshot shows the Google Cloud Logs Explorer interface. At the top, it says 'Google Cloud' and 'WyckoffTrader-Pro by WZ'. There's a search bar and navigation links like 'Query library', 'Share link', and 'Preferences'. The main section is 'Logs Explorer' with filters for 'Project logs', 'Cloud Run Revision +2', 'All log names', 'All severities', and '+5 filters'. Below the filters, there's a timeline view showing a single log entry at 00:04:00. The log entry is an error with the following details:

```

1 error_groups.id="CkqF4cGsu0XngE"
2 logName:"run.googleapis.com/2frequests"
3 resource.type="cloud_run_revision"
4 resource.labels.location="us-central1"
5 resource.labels.configuration_name="papertradesignaltelegramupdate"
6 resource.labels.container_name="wyckofftrader-pro"

```

Below the log entry, there's a table with 7 results. The first result is an error log with the following details:

Severity	Time	Summary
ERROR	2026-09-02 00:33:28.543 MYT	POST 500 8 ms API-Google: https://papertradesignaltelegramupdate-pnjswo3oa-uc.a.run.app/7_GCP_CloudEventsMode=CE_PUBSUB_BINDING
ERROR	2026-09-02 00:33:29.462 MYT	POST 500 8 ms API-Google: https://papertradesignaltelegramupdate-pnjswo3oa-uc.a.run.app/7_GCP_CloudEventsMode=CE_PUBSUB_BINDING
ERROR	2026-09-02 00:33:38.441 MYT	POST 500 8 ms API-Google: https://papertradesignaltelegramupdate-pnjswo3oa-uc.a.run.app/7_GCP_CloudEventsMode=CE_PUBSUB_BINDING
ERROR	2026-09-02 00:33:38.637 MYT	POST 500 8 ms API-Google: https://papertradesignaltelegramupdate-pnjswo3oa-uc.a.run.app/7_GCP_CloudEventsMode=CE_PUBSUB_BINDING

Below the table, there's an 'Investigate log' button and a 'Copy' button. The log entry details are expanded, showing the following information:

```

{
  errorGroups: [1]
  httpRequest: {9}
  insertId: "6a9fe908e39fee8df329"
  labels: {4}
  logName: "projects/wyckofftrader-pro/logs/run.googleapis.com/2frequests"
  receiveTimestamp: "2026-09-01T16:33:46.837627169Z"
  resource: {2}
  severity: "ERROR"
  spanId: "2865208d65277ef"
  textPayload: "The request was aborted because there was no available instance. Additional troubleshooting documentation can be found at: https://cloud.google.com/run/docs/troubleshooting#abort-request"
  timestamp: "2026-09-01T16:33:38.6378252Z"
  trace: "projects/wyckofftrader-pro/traces/8d7863fda804cbb67924559c38f9d9"
}

```

Figure 3 — Firestore data

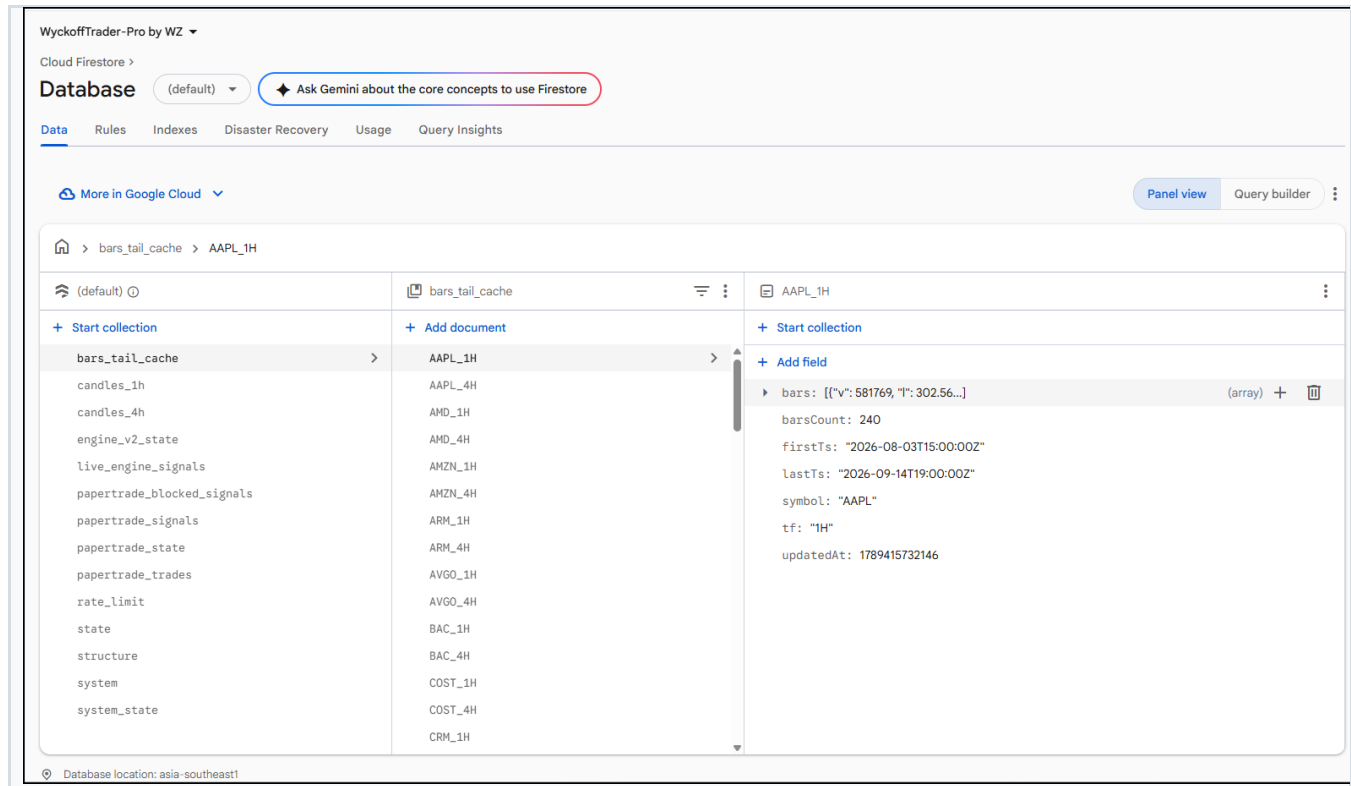


Figure 4 — Example of the signal-to-paper-trade lifecycle, from signal generation to order execution status.

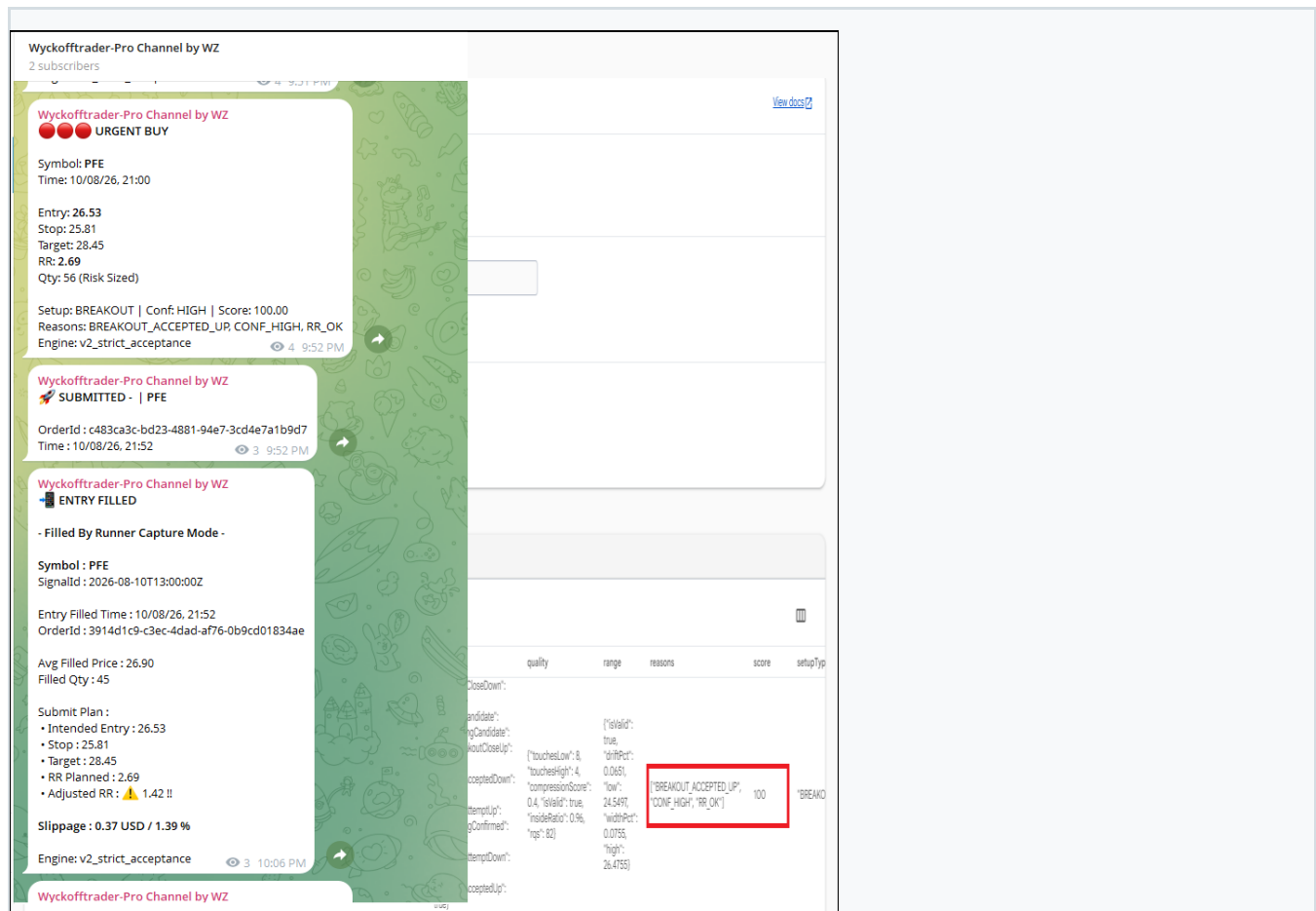


Figure 5 — Telegram Channel & Bot – For notification and report

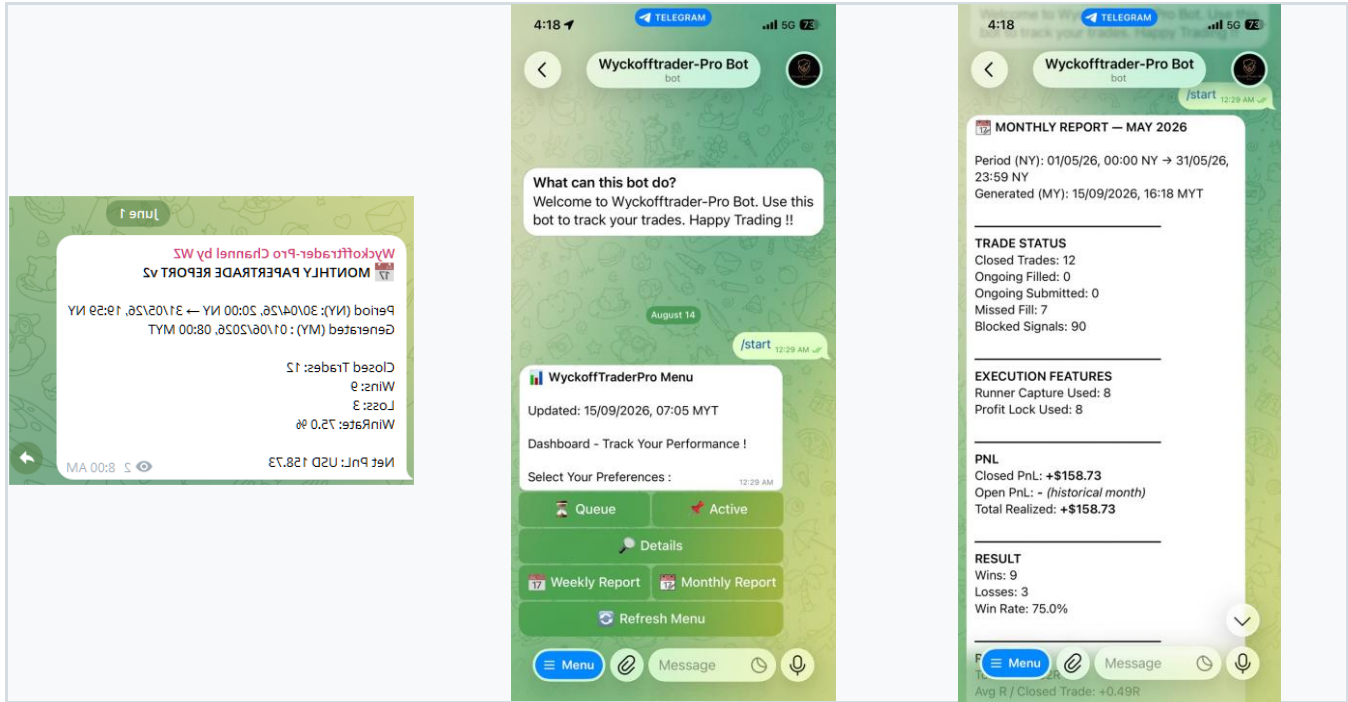


Figure 6 — GitHub portfolio — Insert screenshot of the public portfolio repository.

[INSERT SCREENSHOT HERE]

10. GitHub Repository Presentation

The public GitHub repository is designed as an engineering case study. It contains documentation and selected evidence, while the main application source code remains in a separate private repository.

Repository item	Purpose
README.md	Project summary, architecture, skills, screenshots, limitations and lessons learned.
docs/	Portfolio PDF, architecture notes, troubleshooting and deployment notes.
screenshots/	Selected visual evidence with sensitive information removed.
diagrams/	Architecture and data-flow diagrams.
Private source repository	Main application code, internal logic and sensitive configuration.
Security rule: Never publish API keys, service-account JSON, Telegram bot tokens, trading credentials, private environment files or sensitive database exports.	

11. Project Status & Limitations

WyckoffTraderPro is an evolving personal engineering project. The workflow supports market-data processing, signal generation, paper-trading operations, order-status synchronisation, reporting and operational troubleshooting.

- The public repository contains documentation rather than the full implementation.
- The main source code is maintained in a separate private repository.
- Trading and backtesting outcomes depend on data quality, assumptions and market conditions.
- Some advanced strategy and Wyckoff improvements may remain in shadow or backtest evaluation before being promoted to a baseline.
- The project is presented primarily as an engineering, automation and operations case study.

12. Disclaimer

This is a personal engineering and trading-automation project. Any trading objectives, strategy results or backtesting figures are provided for engineering and analysis context only. They are not financial advice or a guarantee of future trading performance.

Visual Architecture & Workflow Diagrams

The diagrams below provide a simplified, portfolio-friendly view of the system. They intentionally omit private implementation details and credentials.

Input

Market candles

Processing

Serverless backend

Output

Signals and logs

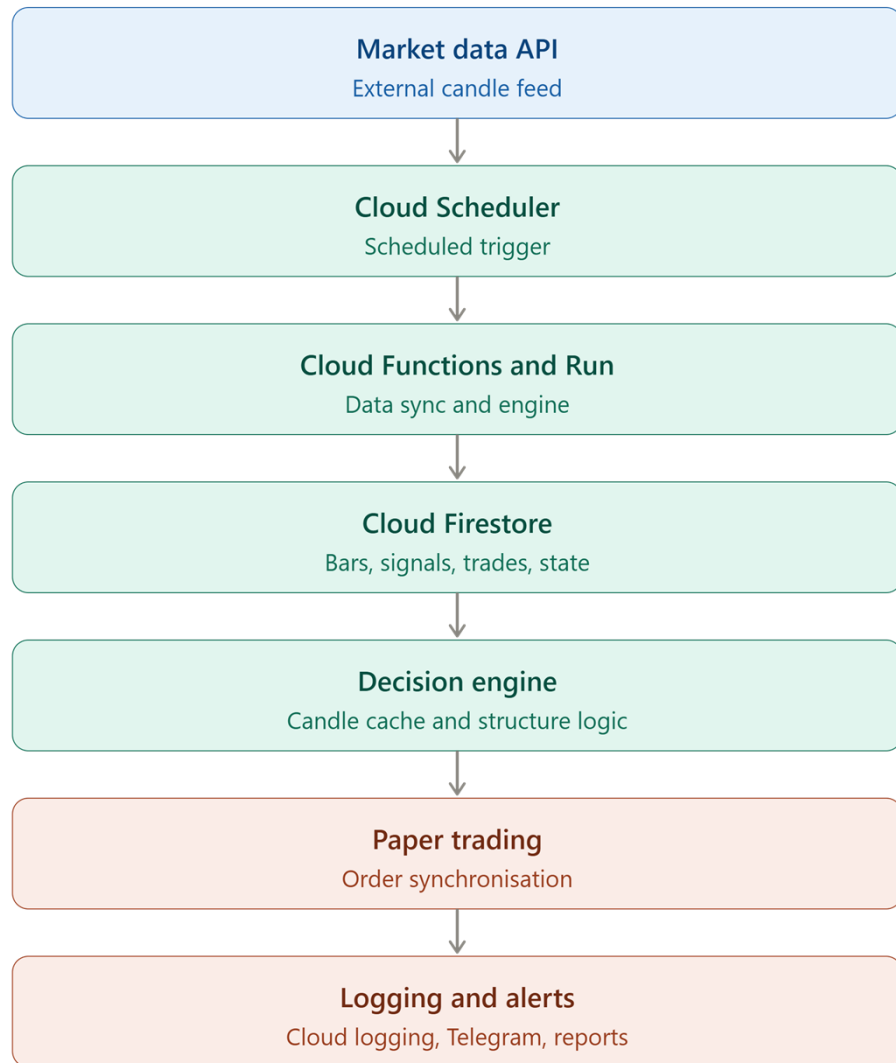
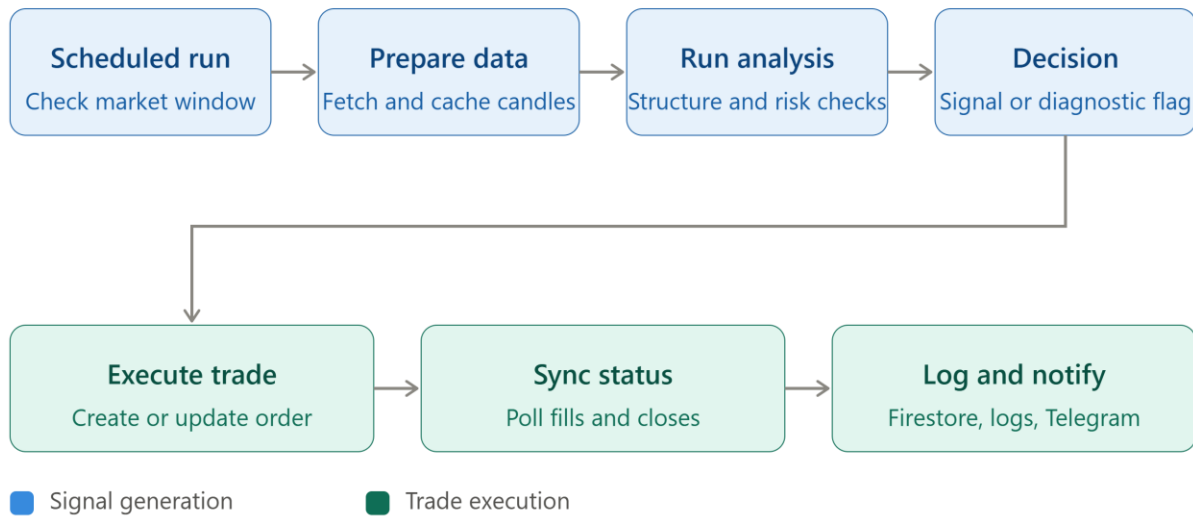


Figure A — High-level cloud architecture



Controls: idempotency, duplicate prevention, bounded retries, rate limiting, diagnostic logging

Figure B — Signal and paper-trade workflow